



OBJETIVOS

- Instalar y configurar los tres entornos de desarrollo disponibles para el MCU RP2040: Arduino IDE, MicroPython y Visual Studio Code (SDK de C/C++ para RP2040).
- Desarrollar un proyecto sencillo en cada entorno que permita controlar el LED integrado de la tarjeta Raspberry Pi Pico y generar un parpadeo con una frecuencia configurable desde el código.
- Familiarizarse con las diferencias y similitudes entre los entornos de programación en cuanto al flujo de trabajo, la sintaxis y la depuración.
- Desarrollar competencias en el uso de herramientas de software y hardware para la programación de microcontroladores en C y en entornos de scripting (Python).

DESCRIPCIÓN

En esta práctica el estudiante trabajará con la tarjeta de desarrollo Raspberry Pi Pico, basada en el microcontrolador RP2040. El objetivo es que el estudiante adquiera experiencia inicial en la programación del MCU en tres entornos distintos, comparando su facilidad de uso, flexibilidad y capacidades.

Cada estudiante deberá:

- Instalar y configurar su computador personal con los entornos Arduino IDE, Thonny (MicroPython) y Visual Studio Code, junto con el SDK de C/C++ para RP2040.
- En cada entorno, crear un proyecto que haga parpadear el LED integrado de la tarjeta a una frecuencia específica.
- Configurar la frecuencia de parpadeo como un parámetro editable en el código, de modo que pueda modificarse fácilmente antes de la compilación o la ejecución.

Requisitos previos:

- Conocimientos básicos de programación en C y Python.
- Conocimiento del uso de entornos de desarrollo y de compilación de proyectos.
- Tarjeta de desarrollo Raspberry Pi Pico, cable micro-USB y un computador personal.

PROCEDIMIENTO

Parte A: Configuración inicial

1. Descargar e instalar los entornos de desarrollo:
 - **Arduino IDE** con soporte para la placa Raspberry Pi Pico.
 - **Thonny IDE** con soporte para **MicroPython** en el RP2040.
 - **Visual Studio Code** con el **SDK de Raspberry Pi Pico para C/C++**.
2. Verificar la comunicación entre la Raspberry Pi Pico y el computador.
3. Cargar un programa de prueba mínimo en cada entorno (por ejemplo: "Hola mundo" o una impresión en la consola).

Parte B: Proyecto Blink en Arduino IDE

1. Abrir Arduino IDE y seleccionar la placa **Raspberry Pi Pico**.
2. Crear un programa en C que configure el LED integrado como salida.

3. Implementar un bucle que encienda y apague el LED con una **frecuencia modificable** definida a través de la terminal.
4. Compilar y cargar el programa en la tarjeta, y comprobar su funcionamiento cambiando la frecuencia de parpadeo.

Parte C: Proyecto Blink en MicroPython

1. Conectar la tarjeta y cargar el firmware de MicroPython en la Raspberry Pi Pico (UF2 oficial de la Fundación Raspberry Pi).
2. Abrir Thonny, seleccionar el intérprete **MicroPython (Raspberry Pi Pico)**.
3. Escribir un script en Python que encienda y apague el LED con una frecuencia configurable desde la terminal.
4. Guardar y ejecutar el script en la placa.

Parte D: Proyecto Blink en Visual Studio Code (SDK C/C++)

1. Instalar las extensiones de CMake, C/C++ y el SDK de Raspberry Pi Pico en VS Code.
2. Crear un proyecto básico de C siguiendo la plantilla de “blink” incluida en el SDK.
3. Definir la frecuencia de parpadeo como una variable, y completar el programa para modificar dicha variable desde la terminal.
4. Compilar el proyecto y cargar el código en la Raspberry Pi Pico.
5. Verificar el funcionamiento y comparar el resultado con el de los demás entornos.

Todos los entornos de desarrollo y las aplicaciones de prueba deben estar instalados y funcionales para la siguiente sección de laboratorio.

EVALUACIÓN

La calificación de la práctica se basará en los siguientes criterios:

- **Instalación y configuración de entornos (30%)**
Verificación de que el estudiante tiene correctamente instalados y operativos los tres entornos de desarrollo.
- **Implementación en Arduino IDE (10%)**
Funcionamiento correcto del programa Blink con una frecuencia configurable.
- **Implementación en MicroPython (10%)**
Script funcional en Thonny con frecuencia configurable.
- **Implementación en Visual Studio Code (50%)**
Proyecto en C basado en el SDK, compilado y ejecutado correctamente en la placa.